CHALMERS
EXAMINATION / TENTAMEN

Course code/kurskod	Course name/kursnamn			
DIT349	Fundamentals of Software Architecture			
Anonymous code Anonym kod		Examination date Tentamensdatum	Number of pages Antal blad	Grade Betyg
DIT349 - 0022 - WSZ		2025 - 10 - 27	9	5

* I confirm that I've no mobile or other similar electronic equipment available during the examination.
Jag intygar att jag inte har mobiltelefon eller annan liknande elektronisk utrustning tillgänglig under eximinationen.

Solved task Behandlade uppgifter No/nr		Points per task Poäng på uppgiften	Observe: Areas with bold contour are to completed by the teacher. Anmärkning: Rutor inom bred kontur ifylles av lärare.
1	✓	18.5	
2	✓	16	
3	✓	19	
4	✓	18	
5	✓	23	
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			
17			
Bonus poäng			
Total examination points Summa poäng på tentamen		94.5	

[redacted]

5

CHALMERS | Anonymous code | Points for question (to be filled in by teacher) | Consecutive page no. Löpande sid nr 1

Anonym kod DIT349-0022-WSZ | Poäng på uppgiften (ifylles av lärare) | Question no. Uppgift nr P1

3P

A. Functional requirement: describes the features of the system WHAT should the system do. 1

Quality attribute requirement: describe the "goodness" of a feature, provide measures to a functionality, should be testable. 1

Constraints: decisions that are premade and must be satisfied. 1

6P

B. Technical constraints: hardware, technology, protocols, etc. 1 1

Example: "The system must be compatible with iOS 17 or later."

It is architectural because it will impact the architecture by ensuring it supports continuous integration/deployment to adapt any potential changes in future iOS updates. 1

Business constraint: time to market, human resources, product lifetime, etc. 1 1

Example: "The system must have at least 5 years lifetime."

It is architectural because the early architecture design should take into consideration of modifiability in order to prepare for any possible future updates. 1

3P

C. Architecturally significant requirements (ASR) are requirements that have an impact on the architecture and/or have high business/mission value to stakeholders. 2

Having an impact on architecture means the architecture will be different depending on whether this requirement is included or not.

Example of NOT ASR: "The UI display shall support red text color." 1

It is implementation related, thus not architecturally significant.

CHALMERS | Anonymous code: DIT349 - 0022 - WSZ | Points for question (to be filled in by teacher): 3.5P | Consecutive page no. Löpande sid nr: 2 | Question no. Uppgift nr: P1

D. Quality attribute scenario elaborate a quality attribute and provide an imaginary situation for a specific QA. 2.5

Example: Performance QAS

Source of stimulus	A user trying to access a resource
Stimulus	Incoming request from user
Environment	At runtime
Artifact	the system
Response	the request is approved and replied
Response measure	reply within 0.5 seconds

E. Performance vs. Availability 3P

To keep the system available, we might want to introduce some redundancy in the system to prevent the system from failure. However, this redundacy might cause computational overhead in the system and decrease performance.

5

A. Definition: A tactic is a design decision that influences the achievement of a quality attribute response. ✓

A tactic can be used to prompt the strength of an architecture style, but it can also refine / cover the weakness in a style by focusing on helping a single QA. ✓ 2 points

B. Split Module
Encapsulate > modifiability

Heartbeat
Active redundancy > Availability

Discover Service
Orchestrate > Interoperability

Detect Intrusion
Authenticate Actors > Security

Sandbox
Limit non-determinism > Testability ✓ 10 points

C. Pipe-and-filter, because it always have 1 quantum.

Every thing needs to be deployed at once for the system to work. All filters and the pipeline needs to be deployed all together. ✓ 2 points

D. Layered Architecture, because it is simple, low cost to build and portable. Making it a perfect choice for simple and small systems. ✓ What about the layered part? ↳ 2 points

Great work!
16 points

CHALMERS | Anonymous code: DIT349 - 0022 - WSZ | Points for question (to be filled in by teacher) / Poäng på uppgiften (ifylles av lärare): 4 | Consecutive page no. / Löpande sid nr: 4 | Question no. / Uppgift nr: P3

A. microservices.

Because, according to the description, there will be a range of different services provideol to users. For example, authentication service to differentiate sellers and buyers, chatting service to support live chat, payment servile to ensure secure payment, etc. These services do not neccessarily share / operate on the same data, thus data isolation is supported.

Overall, microservices fits the best for this system.

The style also supports any future addition / expansion of services because it is highly modifiable and scalable.

GOOD ~~DE~~ MOTIVATION

4/4

CHALMERS

Anonymous code

Anonym kod

DIT349-0022 - WSZ

Points for question (to be filled in by teacher)

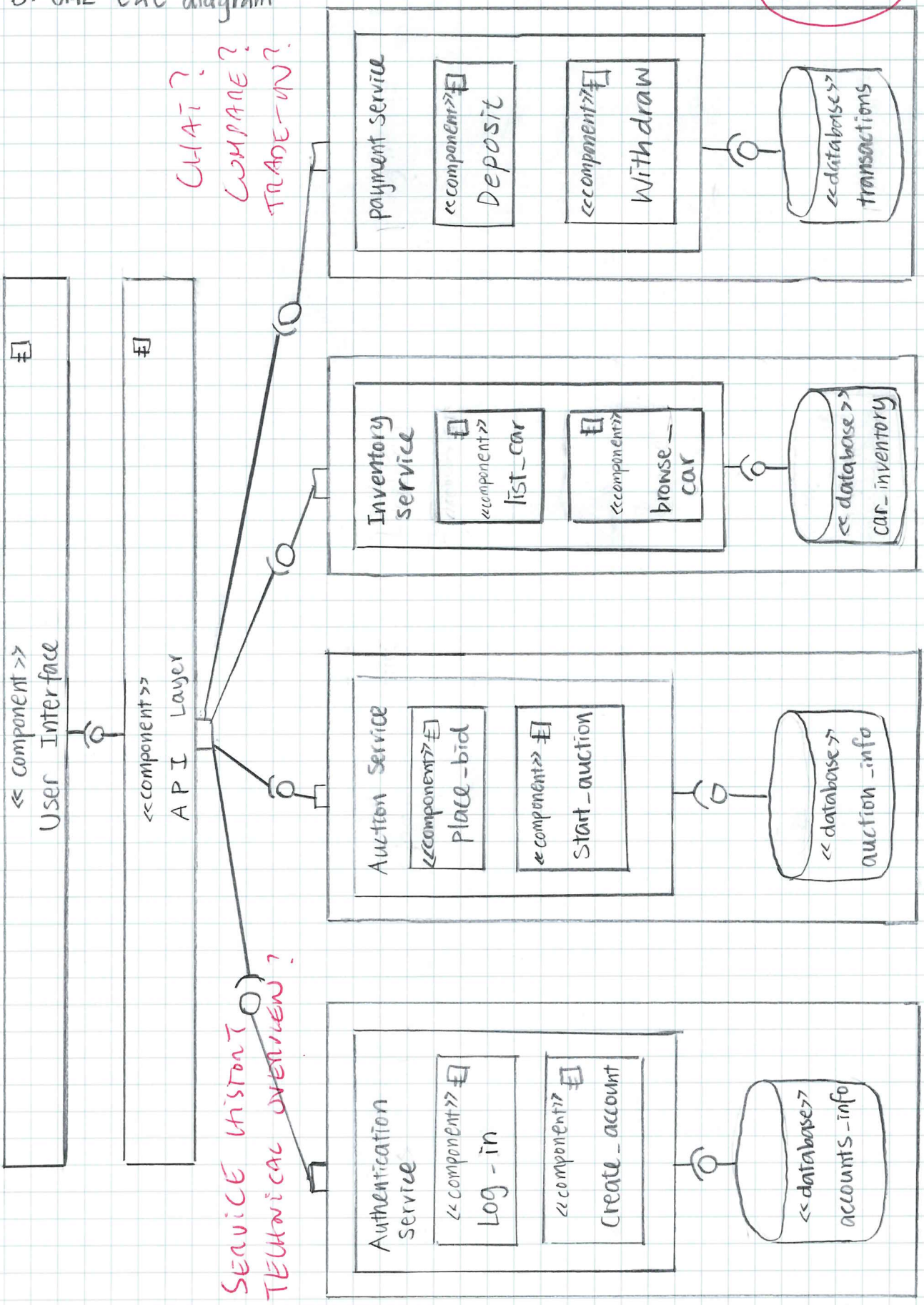
Poäng på uppgiften (ifylles av lärare) 12

Consecutive page no. Löpande sid nr 5

Question no. Uppgift nr P3

12/15

B. UML C&C diagram



* Due to space limit, only 4 services are drawn. There could be more!

5

C. 3 QAs

→ Security : It is an online trading platform, it is always good to have a secure system when it comes to money and trading.

Tactic : Authenticate actors

1p

To authenticate users (login system) before they can trade.

→ Availability : to make sure the system is available during trading. No one would want a system to crash mid-payment.

Tactic : Monitor ? NOT VISIBLE ON THE DIAGRAM

A monitor component detect & monitor activities of different components in the system, allow early discovery of signs of failure.

1p

→ Performance : the platform provides live chat service, we want the messaging service to be efficient, low latency, so that the communication is smooth, and the user don't have to wait forever for a message to be sent or response to show up.

Tactic : Introduce concurrency

Introducing concurrency allow tasks that do not depend on each other run parallel, results in better performance

BUT MICROSERVICES HAVE PERFORMANCE ISSUES

1p

5

A. Information hiding ✓ 1

B. On line 2 - 7, all attributes in "Bank Account" are public. ✓ This violates information hiding. These attributes should be private and the external components should not have access to them!

On line 22 - 26, the client is trying access and modify the attributes of "Bank Account account" and we can see a problem already. On line 22, client can just change the account balance to whatever they want. ✓ On line 24, the client is trying to set the transaction count to an invalid value. ✓ On line 25, the client is basically hacking the transaction history because they can access the whole list! ✓ On line 26, they can set a very large withdraw limit for themselves. ✓ All these actions violate the principle of information hiding which a module should be private to the external environment and should be accessed using interfaces. 5

C. Change line 2-7 to "private" so that external parts cannot directly access them. ✓ Create getters and/or setters for the attribute that client should have access to. ✓ This way we achieve information hiding for the class Bank Account.

Then, replace line 22 - 26 using the newly created getters and setters in Bank Account to ensure client cannot access the private attributes directly. ✓ Some attribute, such as balance, only needs a getter, no setter. ✓ Because client should be able to view the balance, but not modify it.

3

5

P. Seperation of concern (SoC) 1

E. This code violates the SoC principle which states one class / one method should focus on one goal and should have only 1 reason to change. Single responsibility is a specific case of SoC.
The method "handleUserRegistration()" violates SoC. From line 8 - 15, we are taking input from user. Line 16 - 23, we are validating the inputs. Line 24 - 34, we are writing to database. On line 35 - 42, we are printing response messages to user. These parts could have been seperated into smaller, individual methods and make "handleUserRegistration()" more concise and is only responsible for 1 thing. By doing so, we can also make the catch block look cleaner instead of catching multiple different errors in one method. (line 43 - 53) 5

F. First, seperate the functionalities for validating user input, interfacing with database, printing response to user into different methods. We can rename "handleUserRegistration" to "CreateAccount" and from there make function calls to the smaller helper functions that we just created.
Then, inside each method create its own try-catch block, in this way, we can clean up line 43 - 53 in the original code. 3

* Some extra thoughts: I think this example violates SoC and suggested a solution. I don't think my solution leads to high coupling and low cohesion. Because 1. all function calls happen inside class UserManager, there is low coupling to the external factor. 2. High cohesion because all the logic needed for User registration is kept together, not scattered in different components.

5

P5 fill in the gaps

A. 1) structures 2) elements 3) relations ✓

B. static ✓

C. implementation ✓

D. 1) functional 2) quality 3) constraints ✓

E. outside (or beyond ?) ✓ Both work 😊

F. design ✓

G. Availability ✓

H. Interoperability ✓

I. Security ✓

J. Usability ✓

K. components ✓

L. port ✓

M. 1) static ✓ 2) dynamic ✓ 3) ~~allocation~~ physical

N. Allocation ✓

O. interface ✓

P. 1) discovery 2) Architecture ✓

Q. information hiding ✓

23 points